runtime on any machine where you want to deploy the application. Containerizing the application means it will be self-contained and portable across any cloud environment. With containers you can manage all applications in a standard way, no matter the language or framework used to implement them.

The *Open Container Initiative* (OCI), a Linux Foundation project, defines industry standards for working with containers (https://opencontainers.org). In particular, the OCI Image Specification defines how to build container images, the OCI Runtime Specification defines how to run those container images, and the OCI Distribution Specification defines how to distribute them. The tool we'll use to work with containers is Docker (www.docker.com), which is compliant with the OCI specifications.

Docker is an open source platform that "provides the ability to package and run an application in a loosely isolated environment called a container" (https://docs.docker.com). *Docker* is also the name of the company behind this technology, which is a founding member of the OCI. The same term is used in several of their commercial products, too. Whenever I write *Docker*, unless specified otherwise, I am referring to the open source platform we'll use to build and run containers.

Before moving on, follow the instructions in section A.2 of appendix A to install and configure Docker in your development environment.

2.3.1 Introducing Docker: Images and containers

When you install the Docker platform on your machine, you get the Docker Engine package characterized by a client/server architecture. The *Docker server* contains the *Docker daemon*, a background process responsible for creating and managing Docker objects like images, containers, volumes, and